

pyfs-matrix cheatsheet

Typical flow: `inventory` (new mesh) → `plan` (mandatory, no solver) → `run` (`--resume` to add points) → `collect` (cluster jobs) → `post` | **Safe by default:** storage, sync, restore, rebuild and mark-failed only PREVIEW until `--apply`. `--workspace` defaults to the current folder; `--runs NAME` picks another manifest than `runs.json`; a matrix name is found in the workspace root or `inputs/matrices/`.

1. PREPARE THE WORKSPACE AND THE MATRIX

`inventory <geometry>` write `<stem>.boundaries.toml` beside a saved simulation (`.fsm`, from its mesh block) or an OBJ (from its groups); names the solver actions saved in a `.fsm`; no executable needed

- `--overwrite` rewrite an existing `.fsm` sidecar (an OBJ sidecar is never rewritten)
- `--clean` remove the saved solver actions (26.120 in metres: also reset to a fresh import, meshes and BCs kept); copy kept as `<file>.bak-<stamp>`

`upgrade <matrix>` convert a matrix written under an older layout

- `--in-place` rewrite the file (default: print to stdout)
- `--inputs DIR` also move `inputs/groups/e*.toml` into `pproc [groups]`; needs `--in-place`

`rename` rename a `0.20.x` workspace to the `0.21.0` point names (folders, scripts, manifest, plan)

- `--workspace DIR` `--dry-run` print what would move, change nothing

`convert <matrix>` emit the equivalent native `campaign.toml`

- `--fs-exe PATH` **required:** FlightStream executable (never guessed)
- `--name NAME` campaign name (required here: no workspace)
- `--fs-version V` build for rows with empty `FS_BUILD`, e.g. 26.124
- `--recipe CODE=MODULE:FUNCTION` `FS_SCRIPT` code to recipe (repeatable)
- `-o`, `--output FILE` write here instead of stdout

2. PLAN (MANDATORY BEFORE RUN, NO SOLVER)

`plan <matrix>` bind the matrix to `inputs/`, pre-flight every point, write the receipt `run` requires (pinned to the matrix digest; edit the matrix, plan again). Warns, never refuses, on saved actions in an unsteady geometry and on families a row lacks. Alias: `inspect-setups`

- `--workspace DIR` campaign root with `inputs/`
- `--name NAME` campaign name (default: workspace folder name)
- `--fs-version V` fallback build for empty `FS_BUILD` cells
- `--fs-exe PATH` executable override; mandatory for MANUAL rows (else `FS_BUILD` via `inputs/executables.toml`)
- `--workflow CODE=NAME` `FS_SCRIPT` code to run type (repeatable)
- `--recipe CODE=MODULE:FUNCTION` LEGACY recipe (repeatable)
- `--point-name TPL` naming template; default `P<sim>-<point name>`; placeholders `{point}` `{alpha}` `{beta}` `{mach}` `{advance_ratio}` `{sim}` `{campaign}`
- `--ignore-missing-families [true|false]` default true: a family the mesh lacks is skipped; false refuses the point
- `--no-ignore-missing-families` same as false
- `--cost` table mesh size, TEs, farfield layers, viscous coupling, steady/unsteady, iterations, CPUs and an EXPECTED wall time fitted on this workspace's runs
- `--inflow-fft` for `qsteady_rotor` wheel points in a custom inflow: blade-frame harmonics, k_{eff} , n_{max} , suggested `PASSAGE_POSITIONS` (warns if too few)
- `--update-ids` (`--updateIDs`) REWRITE this matrix so no POL repeats one used elsewhere; written before the plan runs
- `--accept-unregistered-build` allow an installed build other than the registered one (your responsibility; recorded)
- `--setup-guidelines` write `inputs/setups/SETUP_GUIDELINES.md`
- `--setup-standards` write commented `s9XX` setup standards
- `--verbose` full warnings (all detail also in `logs/activity.log`)

pyflightstream **0.33.0** every subcommand and option, by stage pyfs-matrix `<cmd>` `-h` (`--help`) gives the full text

3. RUN

`run <matrix>` run every active point and write the sweep table. Needs a current plan. Linux with a submission profile submits to the cluster; Windows runs locally

- `--resume` skip points already recorded (a grown matrix runs only new points); without it a recorded point refuses
- `--force-rerun POINT REDO` a recorded point (by point name, `run_id` or steady JOB id; repeatable). Old records and outputs are archived, nothing deleted. Not with `--resume`
- `--force-rerun-all REDO` every recorded point (prints the licence count first)
- `--sims SIM [SIM ...]` with `--force-rerun-all`: only these simulations, e.g. `--sims 2031 2032`
- `--local` run on THIS machine instead of submitting (Linux workstation or smoke test)
- `--progress-every N` local unsteady progress every N steps (default 10; 0 = silent)
- `--sweep-csv FILE` sweep table path (default `post/<matrix>/campaign_sweep.csv`)
- `--workflow CODE=NAME` run type: `steady`, `unsteady`, `unsteady_rotor`, `qsteady_rotor`
- `--workspace`, `--name`, `--fs-version`, `--fs-exe`, `--recipe`, `--point-name`, `--ignore-missing-families`, `--no-ignore-missing-families`, `--accept-unregistered-build`: as in `plan`
- `--pproc-warnings` print grouped post-processing warnings
- `--verbose` full warnings and one line per repeated item

4. COLLECT (CLUSTER OR INTERRUPTED RUNS)

`collect` for every SUBMITTED record, wait until its outputs are present and SETTLED, collect, assess, update the record, then post; HPC job_end_files present, no solver log: `FAILED_EXECUTION`

- `--watch` keep sweeping until nothing is outstanding (default: one sweep)
- `--watch-interval S` seconds between sweeps (default 60)
- `--interval S` seconds between the two size/time observations (default 2)
- `--rounds N` stop a watch after N sweeps
- `--sims IDS` only these simulations, e.g. 2006,2007, and their post
- `--no-post` collect only, do not rebuild products
- `--check-frozen REFUSE` (not warn) averages touched by a frozen solve or an unread log block
- `--workspace`, `--runs NAME`, `--pproc-warnings`, `--verbose`

Tip: `nohup pyfs-matrix collect --watch > logs/collect.out 2>&1 &`

5. POST-PROCESS (NO SOLVER, NO SEAT)

`post [matrix]` rebuild polars, sections and plot tables under `post/<matrix>`; no matrix = every matrix in the manifest. Old products are archived in `post/<matrix>/archive/`

- `--sims IDS` only these simulations' products, in place (super files kept)
- `--runs NAME` read another manifest (e.g. a rebuilt one)
- `--from-sims` ignore `runs.json`, build records in memory from `sims/` (runs made outside the package); products go to `post/<matrix>@sims/`
- `--steps-per-revolution N` with `--from-sims`: for a `LAST_REVS_AVG` window
- `--additional-pproc` reopen the final `.fsm` of rows with `ADDITIONAL_PPROC` (no solve, one launch per point) and extract that `pproc`; 26.124 only. With it: `--fs-version`, `--fs-exe`, `--local`, `--recipe`
- `--diagnostics` print recorded post diagnostics as Markdown, change nothing
- `--strict` exit 3 when a product was skipped by design (e.g. polar under sideslip)
- `--check-frozen` refuse instead of warn (see `collect`)
- `--force-overwrite` DESTROY products instead of archiving (asks; `--yes` answers it)
- `--workspace`, `--pproc-warnings`, `--verbose`

6. SYNC WORKSPACES (E.G. CLUSTER TO WINDOWS)

`sync {runs|post|fsm|all}` run on the MAIN workspace; peers in `inputs/sync-workspaces.toml`; levels cumulative, `runs < post < fsm < all`. Nothing in main is deleted; `inputs/` untouched but a declared matrix

- `--apply` change files (default: preview)
- `--from NAME` one peer (default: every non-main one)
- `--prefer-other` on a record conflict take the peer's (default: main wins)
- `--overwrite` on a file conflict archive main's copy and take the peer's
- `--restore` with `--apply`: rebuild records of `sims/` folders no record carries
- `--include-archives` also bring archive folders (skipped by default)
- `--workspace`, `--runs NAME`

7. RECOVER AND MARK RECORDS

`restore {runs|storage|products|plan|additional}` restore a file from archive/

- `--apply` (default: preview) `--stamp STAMP` archive to restore from
- `--matrix STEM` for products and plan, when several matrices have archives
- `--workspace`

`rebuild` rebuild run records from the folders under `sims/`

- `--apply` (default: preview) `--out NAME` write records to another manifest (compare before replacing)
- `--all-sims` every simulation folder, recorded or not `--sims 4001,2009` only these
- `--matrix FILE` the matrix the simulations ran from
- `--build-alias BUILD=ALIAS` scheduler name of a build (repeatable), e.g. 26.124=26.12
- `--inputs-from FOLDER` another origin's `inputs/` for inputs changed after the run
- `--workspace`

`mark-failed` every record of the named simulations becomes `FAILED_MARKED`, its old status kept under marked; `runs.json` archived first. Not readable before 0.33.0

- `--sims 2006,2007` the simulations `--reason TEXT` why, recorded as given
- `--apply` (default: preview) `--workspace`

8. DISK SPACE (EVERY CALL LOGGED IN storage_management.json)

`space-in-use` sizes by top folder, by `sims/sim_*`, by extension; changes nothing

- `--top N` rows per grouping (default 15) `--workspace`

`free-space <m-id>` run storage recipe `inputs/management/m<id>.toml`: `prune_step_exports`, `compact_sims` (zip, restored automatically), `delete_extensions`, `post_archives`

- `--apply` (default: preview) `--runs NAME` `--workspace`
- `--list` every path each step touches, with its size and what happens to it

`delete-sims <ids>` delete simulations, their products and records, e.g. 4001,2009; a note row remains

- `--apply` (default: preview) `--runs NAME` `--workspace`
- `--force` also a simulation whose run is still SUBMITTED (refused without it)
- `--matrix-products {points-only|regenerate}` for a matrix product that also holds the deleted points: leave it (marked stale) or rerun its post without them

Before a licensed run: `plan` with `--cost`; read the warnings; `inventory` `--clean` a geometry that carries saved actions. **Exit codes of post:** 0 done; 2 refused, nothing written; 3 with `--strict`, a product skipped by design. **Trace:** `runs.json`, `plan.json`, `post/`, `logs/`.